Maîtrise de la Syntaxe JSX et Rendu Dynamique

Objectifs

1. Convertir du balisage HTML statique brut en code JSX valide respectant les contraintes strictes de React.
2. Utiliser des fragments (`<> ... </>`) pour éviter l'accumulation de balises conteneurs superflues dans le DOM.
3. Injecter et évaluer des expressions JavaScript dynamiques au sein des balises JSX.
4. Implémenter des rendus conditionnels propres à l'aide d'opérateurs ternaires et logiques.
5. Transformer dynamiquement des structures de données en listes d'éléments JSX à l'aide de la méthode `.map()`.
6. Appliquer des styles visuels sous forme d'objets JavaScript en camelCase ou via des classes CSS conditionnelles.

Contexte du Projet : OmniMonitor - Tableau de Bord et Alertes

Vous enrichissez l'interface de l'application **OmniMonitor**. L'objectif est de transformer une maquette HTML statique héritée en un composant React moderne entièrement piloté par des variables et des structures de données JavaScript injectées en JSX.

Étape 1 : Nettoyage et Correction Syntaxique d'un Template HTML brut

On vous transmet un vieux fragment de code HTML non conforme que vous devez intégrer et nettoyer dans un composant React.

1. Créez un nouveau composant `src/components/LegacyPanel.jsx`.
2. Reprenez le code ci-dessous et corrigez toutes les erreurs de syntaxe pour qu'il devienne du JSX valide (fermeture des balises orphelines, remplacement des attributs réservés) :

```
None
// CODE NON CONFORME À CORRIGER :
```

```
export default function LegacyPanel() {
  return (
    <div class="legacy-box">
      <h1>Rapport de Maintenance</h1>
      <hr>
      <img src="banner.png">
      <br>
      <label for="comment">Note opérateur :</label>
      <input type="text" maxlength="50" tabindex="1">
    </div>
  );
}
```

3. Vérifiez que votre code ne génère aucune erreur de compilation dans l'éditeur et le navigateur.

Étape 2 : Injection d'Expressions JavaScript Dynamiques

Vous allez afficher des données textuelles et numériques calculées dynamiquement.

1. Dans un nouveau composant `src/components/StationStatusWidget.jsx`, déclarez les constantes suivantes à l'intérieur de la fonction :
    - `stationCode = "TLS-04"`
    - `batteryLevel = 84` (pourcentage)
    - `lastUpdate = new Date()`
    - `operator = { firstName: "Élodie", role: "Superviseur Réseau" }`
2. Rédigez le bloc JSX de retour pour afficher :
    - Le code de la station en majuscules (en utilisant la méthode `.toUpperCase()`).
    - Le niveau de batterie.
    - La date formatée (via `lastUpdate.toLocaleTimeString()`).
    - Le prénom et le rôle de l'opérateur (en accédant aux propriétés de l'objet).

Étape 3 : Rendu Conditionnel d'Alertes Techniques

Vous allez afficher des messages contextuels selon l'état critique de la station.

1. Créez un composant `src/components/AlertBanner.jsx` qui reçoit deux props : `isEmergency` (booléen) et `errorCount` (nombre).
2. Utilisez l'opérateur logique `&&` pour afficher une bannière rouge d'urgence uniquement si `isEmergency` vaut `true`.
3. Utilisez l'opérateur ternaire `? :` pour afficher soit le texte `"Système nominal (" + errorCount + " alertes mineures)"` si `errorCount > 0`, soit `"Aucune anomalie détectée"` si le compteur est à zéro.

Étape 4 : Génération Dynamique de Listes avec `.map()`

Vous devez afficher l'historique des derniers relevés de capteurs sous forme de tableau ou de liste à puces.

1. Dans un composant `src/components/SensorReadings.jsx`, intégrez le tableau de données suivant :

```
None
const readings = [
  { id: 101, type: 'Température', value: '24.5 °C', status: 'normal' },
  { id: 102, type: 'Pression Atmosphérique', value: '1013 hPa', status: 'normal'
},
  { id: 103, type: 'Vitesse du Vent', value: '68 km/h', status: 'warning' },
  { id: 104, type: 'Taux d'Humidité', value: '92%', status: 'critical' }
];
```

2. Utilisez la méthode `.map()` pour parcourir ce tableau et générer une liste de balises `<li>`.
3. **Point critique :** Assurez-vous d'ajouter l'attribut `key={reading.id}` sur chaque élément racine généré par la boucle.

Étape 5 : Gestion Avancée des Styles (Inline et Classes Dynamiques)

Vous allez appliquer des styles dynamiques en fonction du statut des capteurs de l'étape précédente.

1. Créez un objet de style inline JavaScript pour un conteneur principal (fond neutre, bordure arrondie, espacement interne).
2. Pour chaque élément de la liste des capteurs, appliquez dynamiquement une classe CSS conditionnelle (par exemple, si le statut est `critical`, ajoutez la classe `.text-red-bold`, sinon `.text-normal`) en combinant des chaînes de caractères littérales.

Critères de Validation

Avant de valider ce segment, vérifiez que :

- Toutes les balises sans enfants possèdent bien un slash final de fermeture (`<br />`, `<img ... />`, `<input ... />`).
- Aucun attribut HTML interdit (`class`, `for`) ne subsiste dans le code JSX (remplacés par `className` et `htmlFor`).
- Chaque élément généré par `.map()` possède un attribut `key` unique et stable.
- L'application compile sans aucune erreur de syntaxe.